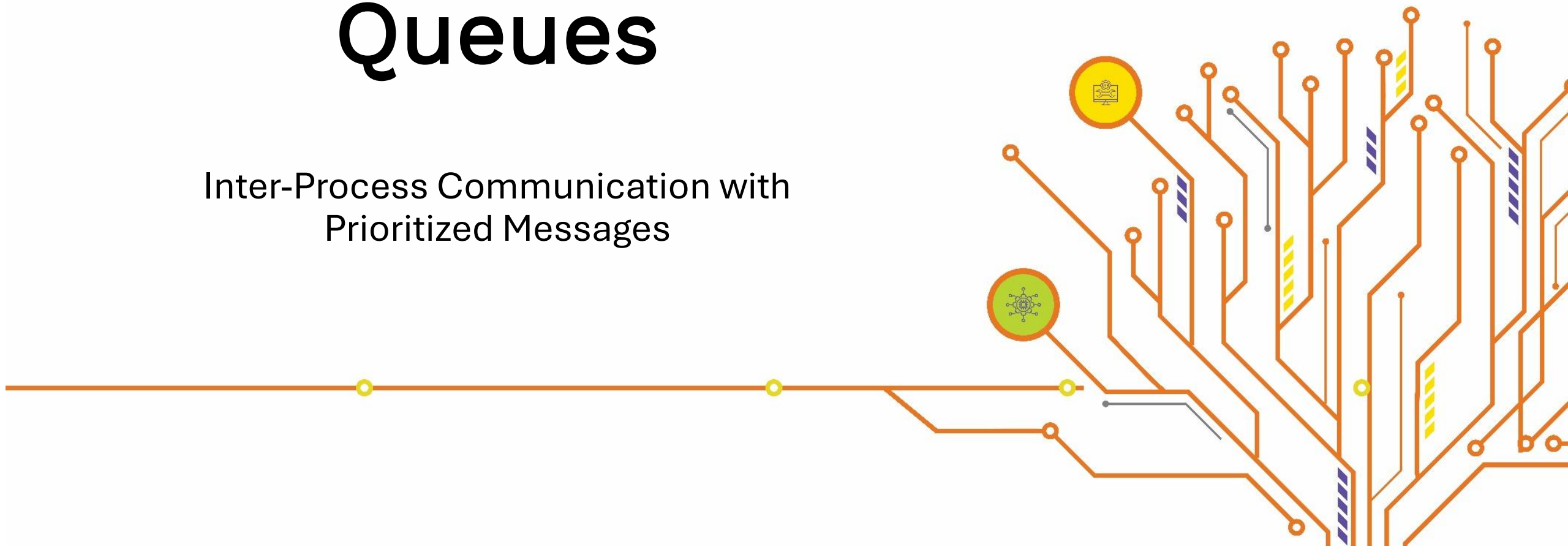
KPIT | APEX LAB

POSIX Message Queues

Inter-Process Communication with Prioritized Messages



Disclaimer

This material has been developed exclusively for educational and academic purposes under the KPIT APEX Lab – Academic CSR Initiative. It is intended strictly for student learning, demonstrations, and classroom use.

It must not be copied, forwarded, shared, reproduced, or redistributed to any other individuals, groups, institutions, or platforms without prior written permission.

All examples, commands, diagrams, and demonstrations included here are meant solely to support conceptual understanding and learning.

Any use of these contents in commercial products, paid workshops, training programs, or proprietary documentation is strictly prohibited.

What is a POSIX Message Queue?

- IPC mechanism that allows processes to **exchange messages**
- Messages have **fixed maximum size** and **user-defined priority**
- Kernel stores messages inside a **managed queue**
- Accessible through a **name** (like /myqueue)
- Faster and simpler than sockets for small, structured messages

Why Use Message Queues?

Need to send structured data, not raw bytes

Want asynchronous communication

Want priorities (urgent messages delivered first)

Avoid complexity of pipes or shared memory

Useful in multi-process embedded applications

(sensor tasks, command/control, alarms, events)

Key Characteristics

- Queue lives inside **kernel**
- Identified by a **name** (/mq_name)
- Supports **blocking & non-blocking** operations
- Supports **synchronous notifications** (signals or threads)
- Message order = by **priority**, then FIFO
- Must be created and configured using **attributes**

Message Queue APIs (mq_xxx)

- mq_open() → create/open
- mq_send() → send message
- mq_receive() → receive message
- mq_getattr() / mq_setattr() → view/modify attributes
- mq_notify() → signal/thread notification
- mq_close() → close MQ descriptor
- mq_unlink() → delete queue

Creating a Queue

```
struct mq_attr attr;
attr.mq_flags = 0;
attr.mq_maxmsg = 10;
attr.mq_msgsize = 64;
attr.mq_curmsgs = 0;

mqd_t q = mq_open("/apexq", O_CREAT | O_RDWR, 0666, &attr);
```

- Name must start with '/'
- Queue persists until mq_unlink()

Sending Messages (mq_send)

```
mq_send(q, "Hello", 6, 2);  // message, length, priority
```

- Priority: 0–31 (higher delivered first)
- Blocks if queue full
- Non-blocking mode → returns EAGAIN
- Message larger than msgsize → error (EMSGSIZE)

Receiving Messages (mq_receive)

```
char buf[64];
unsigned int prio;
mq_receive(q, buf, sizeof(buf), &prio);
```

- Behavior:
  - Blocks if queue empty
  - Non-blocking → EAGAIN
  - Returns length of received message
  - Outputs priority of message

Message Priorities

- Priorities allow **urgency-based** message retrieval
- **Higher priority** messages dequeued **first**
- Within same priority → **FIFO order**
- Useful for:
  - Alarms
  - emergency stop events
  - high-frequency sensor updates

Blocking vs Non-blocking

- **Blocking mode: (default)**
  - mq_send() waits if queue full
  - mq_receive() waits if queue empty
- **Non-blocking mode:**
  - mq_send() → EAGAIN on full
  - mq_receive() → EAGAIN on empty
- **Enable non-blocking:**
  - Use O_NONBLOCK in mq_open()
  - Or change via mq_setattr()

Queue Attributes (Inspection)

```
struct mq_attr info;
mq_getattr(q, &info);
```

- mq_curmsgs → current messages
- mq_maxmsg → capacity
- mq_msgsize → message size limit
- Useful for monitoring & debugging.

Cleaning Up Queues

- mq_close(q); → close descriptor
- mq_unlink("/apexq"); → remove queue from system

Queue persists in kernel until unlink, even if all processes exit.

Always unlink after labs.

Example: Sender and Receiver

```
mqd_t q = mq_open("/apexq", O_WRONLY);
mq_send(q, "SPEED=40", 9, 1);
mq_close(q);
```

```
mqd_t q = mq_open("/apexq", O_RDONLY);
char buf[64]; unsigned int p;
int n = mq_receive(q, buf, 64, &p);
printf("Msg: %s, Priority: %u\n", buf, p);
mq_close(q);
```

separate sender process uses same queue name.

Error Conditions

- Queue full → mq_send → EAGAIN or block
- Queue empty → mq_receive → EAGAIN or block
- Wrong queue name → ENOENT
- Exceeding msgsize → EMSGSIZE
- No permissions → EACCES
- Queue deleted while open → operations fail

Best Practices

- Define a message struct instead of raw text
- Keep messages small (<100 bytes)
- Avoid heavy binary blobs (use shared memory instead)
- Always close + unlink in cleanup
- Use priorities carefully (not everything should be prio 31)

Use Cases

- Sensor → Controller queue
- Event dispatcher
- Logging framework
- Autonomous robot state updates
- Diagnostics & notification events
- Multi-process architectures (ECU services)

Example 1: Basic MQ Send/Receive

Task:

- Create queue /labq
- Write one sender, one receiver
- Send 5 messages
- Receiver prints them in arrival order

Example 2: Using Priorities

- Sender sends messages with priorities 1, 5, 3
- Receiver should print: p5, p3, p1
- Then delete queue

Learning: priority-based ordering

Example 3: Multi-Process Fan-In Queue

Setup:

- 3 producer processes → send events
- 1 consumer → prints all
- Use priorities for severity levels (INFO/WARN/CRITICAL)

Learning: multi-process MQ usage

Common Pitfalls

- Forgetting to unlink queue → leftover kernel objects
- Using too large msgsize → heavy overhead
- Ignoring return values
- Overusing high priorities
- Trying to use MQ for streaming or huge data

Summary

- POSIX MQs provide structured, prioritized message IPC
- Easy for unrelated processes
- Blocking & non-blocking modes available
- Supports notifications
- Great for event-driven systems and modular architectures

Scenario: Prioritized Event Bus for Alerts (1/4)

Context

- Develop an event bus using a named POSIX message queue /apex_events.
- Multiple producers send events (INFO/WARN/CRITICAL) with priorities; a single consumer receives and handles them in priority order.

Scenario: Prioritized Event Bus for Alerts (2/4)

PART 1 — Queue Setup (Consumer)

- Create event_consumer.c.
- mq_open("/apex_events", O_CREAT | O_RDONLY | O_NONBLOCK, 0666, &attr)
  - mq_maxmsg = 16, mq_msgsize = 128
- Loop: mq_receive messages; if none, sleep 100 ms; print:

  [prio=<p>] <timestamp> <source> <msg>

Scenario: Prioritized Event Bus for Alerts (3/4)

PART 2 — Producer(s)

- Create event_producer.c.
- CLI: ./event_producer <prio> <source> <msg...>
- Map prio: CRITICAL=5, WARN=3, INFO=1.
- mq_send(q, payload, len, prio)

Scenario: Prioritized Event Bus for Alerts (4/4)

PART 3 — Notification

- In consumer, register mq_notify (signal or thread).
- On transition from empty→non-empty, wake up and drain the queue immediately.